Modelos de Calidad de Software

Escuela de Tecnología

Mg. Daniel Arias, **PMP™** y **Scaled Scrum Master™**

¿Qué aprendimos la clase pasada?

- Definición de Pruebas de Software
- ¿Cuántas pruebas son suficientes?
- Causas de los defectos de software.
- Ciclo de vida de las pruebas de software
- Los 7 principios de las pruebas de software

idat

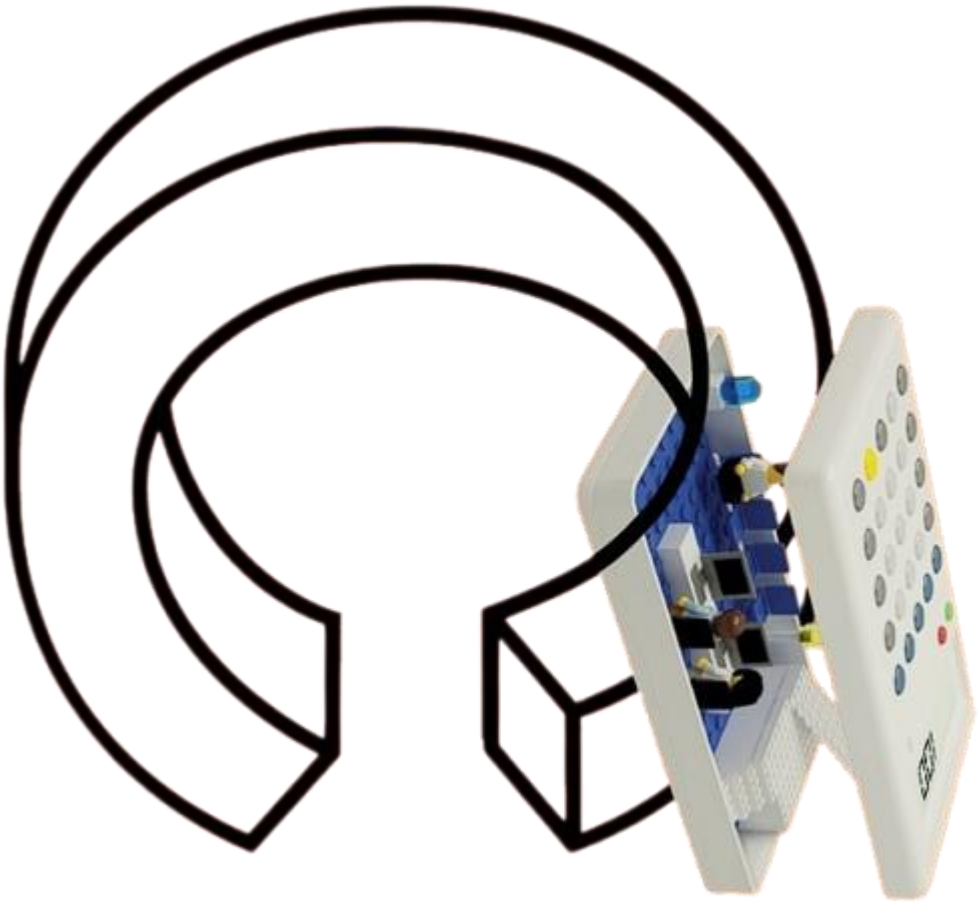
Conversemos

http://www.menti.com

idat

Introducción a Calidad de Software – Parte 4

Semana 4



idat

Logro de aprendizaje

- Niveles de pruebas
- Tipos de pruebas
- Técnica de Pruebas de Caja Negra y Caja Blanca

Actividad de aprendizaje

ACTIVIDAD VIRTUAL

Revisa y analiza el enlace sugerido en la plataforma virtual luego realiza la actividad propuesta

- ¿Cuál es la diferencia entre niveles y tipos de pruebas?
- ¿Qué niveles de prueba efectúa el desarrollador?
- ¿Qué niveles de prueba efectúa el analista de calidad?
- Brindar ejemplos prácticos de Caja Negra y Caja blanca.

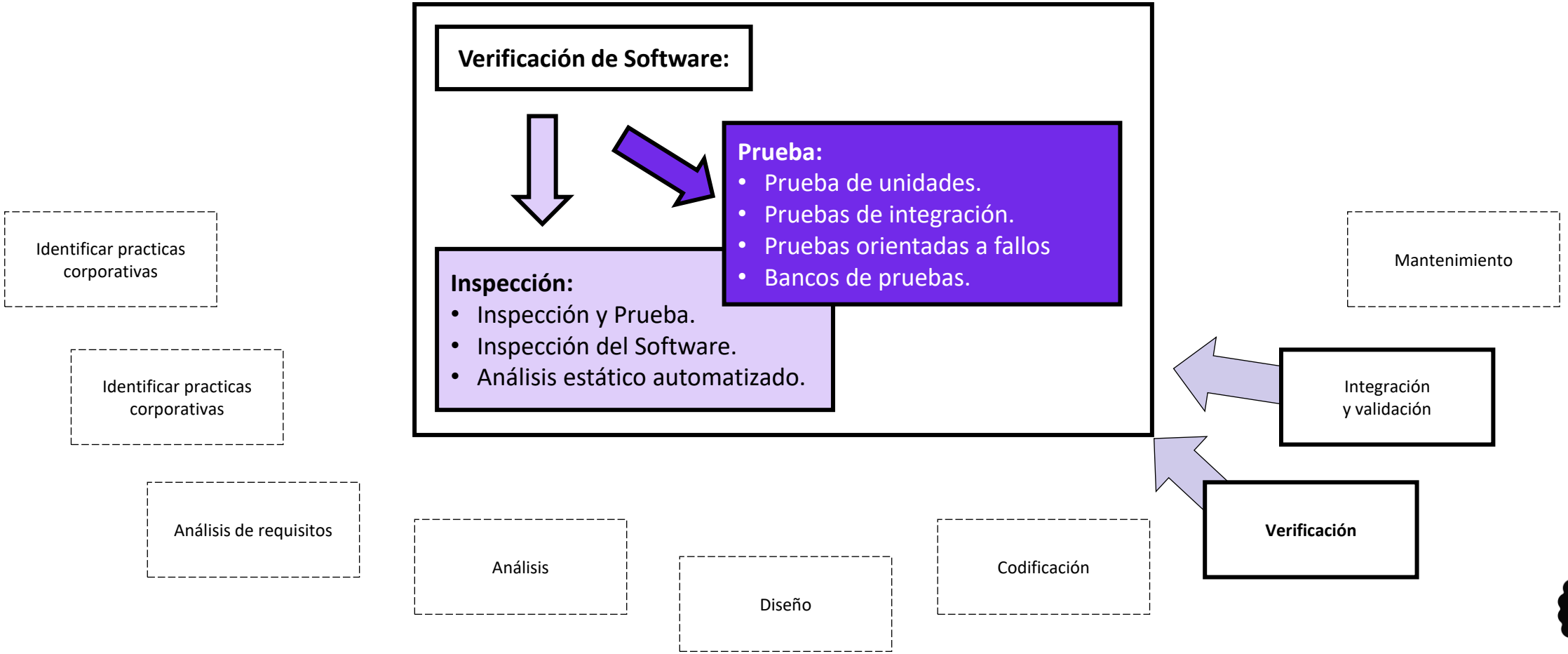
idat

Logro de aprendizaje

Identificar la diferencia entre los niveles y tipos de prueba para aplicarlo según el contexto.

idat

Situación dentro del proceso de desarrollo



idat

Verificación y Validación

Verificación:

¿Estamos construyendo el producto correctamente?

Se comprueba que el software cumple los requisitos funcionales y no funcionales de su especificación.

Validación:

¿Estamos construyendo el producto correcto?

Comprueba que el software cumple las expectativas que el cliente espera.

Con V&V se pretende certificar que el sistema desarrollado se ajusta a lo esperado de la forma mas satisfactoria posible. Nunca se puede demostrar que está libre de defectos

idat

La Estrategia en Capas

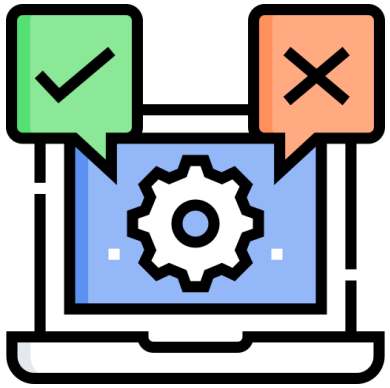
Dividir el proceso de testing en fases es fundamental para hacerlo más manejable, eficiente y efectivo. Esta estrategia en capas asegura que cada componente del software sea robusto, contribuyendo a un producto final de alta calidad.

Detección Temprana
Encontrar errores en la fase más temprana y barata de corregir.

Aislamiento de Defectos
Es más fácil encontrar un defecto en un componente pequeño que en un sistema enorme.

Reutilización
Los casos de prueba de niveles inferiores se pueden integrar en niveles superiores.

Enfoque Diferente
Cada nivel tiene un objetivo y un público específico.



idat

Los Niveles de Pruebas de Software

Garantizando Calidad de Forma Integral y Sistemática

idat

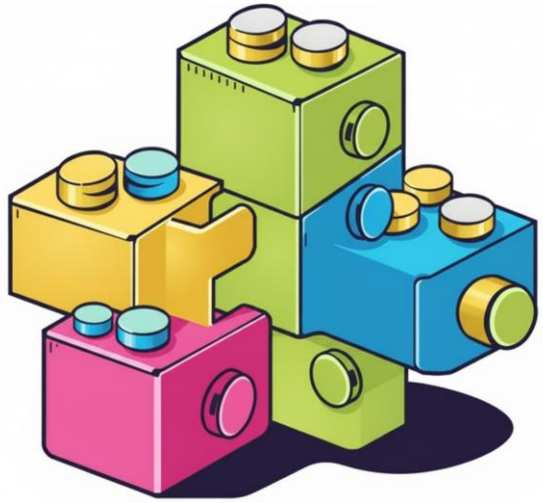
Los Niveles de Pruebas de Software

- **Nivel 1 - Pruebas Unitarias (Unit Testing)**
  Los Cimientos de la Calidad
  Las pruebas unitarias se enfocan en las unidades más pequeñas de código: funciones, métodos o clases, probándolas de forma **aislada**. Son la primera línea de defensa de la calidad.
  - **¿Qué se prueba?** Unidades más pequeñas de código: funciones, métodos, clases.
  - **¿Quién lo hace?** Principalmente los desarrolladores durante la codificación.
  - **Objetivo:** Verificar que cada "ladrillo" del software funcione correctamente por sí solo.
  - **Características:** Rápidas, automatizadas (JUnit, NUnit), cubren lógica interna.

idat

Los Niveles de Pruebas de Software

- **Nivel 2 - Pruebas de Integración**
  ¿Juegan bien juntos?
  Las pruebas de integración validan la interacción y comunicación entre **módulos o servicios** que han sido integrados. Son cruciales para asegurar que los componentes trabajen armónicamente.
  - **¿Qué se prueba?** Interacción y comunicación entre módulos o servicios integrados.
  - **¿Quién lo hace?** Desarrolladores y/o ingenieros de QA.
  - **Objetivo:** Encontrar defectos en las interfaces e interacciones (APIs, bases de datos, servicios externos).



idat

Los Niveles de Pruebas de Software

- **Nivel 3 - Pruebas de Sistema (System Testing)**
  Probando el Producto Completo
  En este nivel, se prueba el **sistema completo e integrado** en un entorno que simula la producción, garantizando que cumple con todos los requisitos. Es una validación integral de la solución.
  - **¿Qué se prueba?** El sistema completo e integrado como un todo.
  - **¿Quién lo hace?** Equipo de Control de Calidad (QA) independientemente del desarrollo.
  - **Objetivo:** Validar que el sistema cumple con requisitos funcionales y no funcionales.

  Tipos de pruebas incluidas:
  - Funcionalidad, Rendimiento (Carga, Estrés), Usabilidad.
  - Seguridad, Compatibilidad, y más.

idat

Los Niveles de Pruebas de Software

- **Nivel 4 - Pruebas de Aceptación**
  La Aprobación Final del Cliente
  Estas pruebas evalúan el sistema desde la perspectiva del **usuario o cliente final**, asegurando que el software resuelve su problema y está listo para producción.
  - **¿Qué se prueba?** El sistema desde la perspectiva del usuario o cliente final.
  - **¿Quién lo hace?** El cliente o los usuarios finales/Product Owners.
  - **Objetivo:** Determinar si el sistema está listo para ser liberado y usado en producción.

Tipos comunes:

UAT (User Acceptance Testing)	Pruebas Alfa/Beta	Pruebas Contractuales
El cliente prueba en un entorno simulado.	Alfa (interna controlada), Beta (usuarios reales externos).	Basadas en criterios de aceptación definidos por contrato.

idat

La Pirámide de Pruebas - Un Enfoque Sólido

La pirámide de pruebas ofrece una guía visual para una estrategia de testing equilibrada, optimizando recursos y maximizando la calidad.

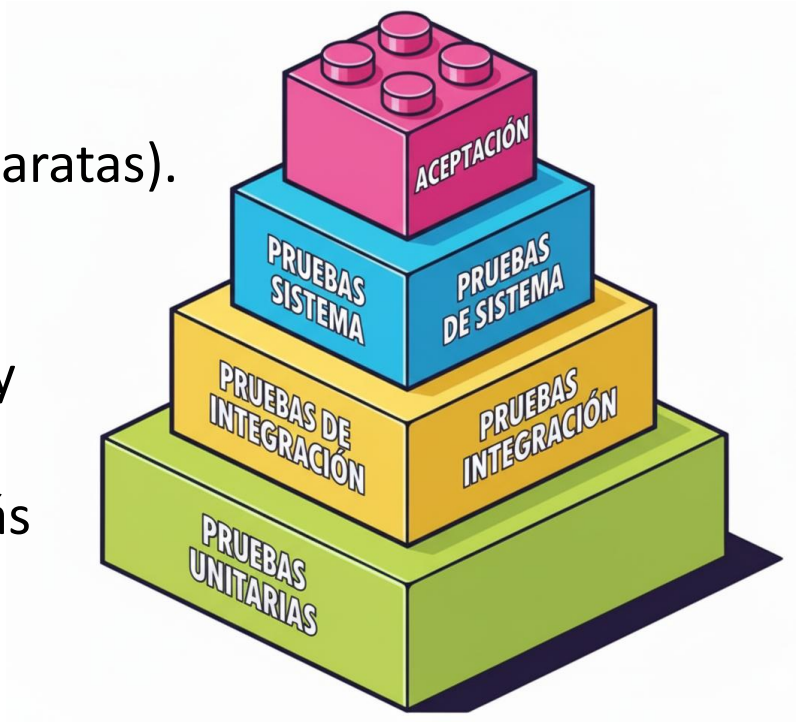
idat

La Pirámide de Pruebas - Un Enfoque Sólido

La pirámide de pruebas ofrece una guía visual para una estrategia de testing equilibrada, optimizando recursos y maximizando la calidad.

La Pirámide Ideal:

- Base Ancha: Muchas **Pruebas Unitarias** (rápidas y baratas).
- Nivel Medio: Un número menor de **Pruebas de Integración**.
- Nivel Alto: Menos **Pruebas de Sistema** (más lentas y costosas).
- Cúspide: Muy pocas **Pruebas de Aceptación** (las más complejas).



idat

Tipos de Pruebas de Software

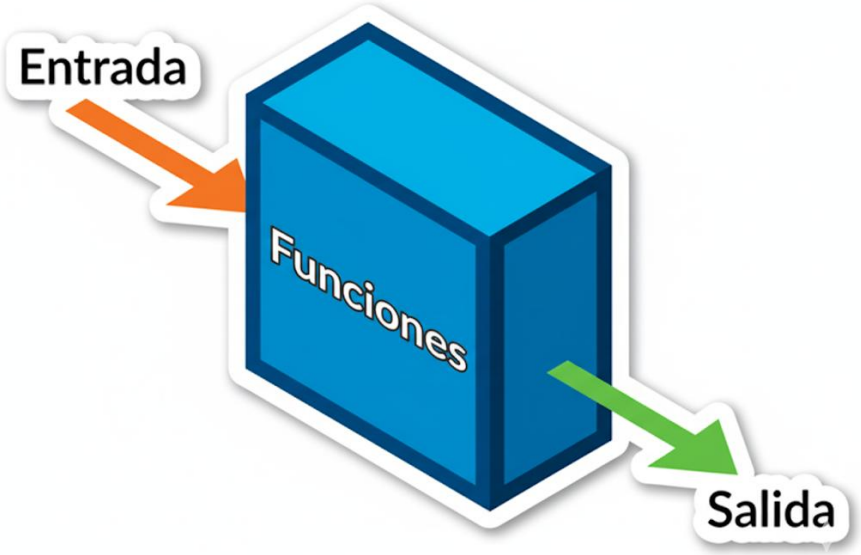
Pruebas funcionales ("Caja negra")

- Las pruebas funcionales o de caja negras es una política de selección de casos de pruebas basado en la especificación del componente o programa.
- Las pruebas se seleccionan en función de la especificación y no de la estructura interna del software.

idat

Tipos de Pruebas de Software

1. **Pruebas funcionales o de caja negra**
   Objetivo: probar la función del objeto de prueba
   - La funcionalidad puede ser vinculada a los datos de entrada y salida de un objeto de prueba.
   - Los métodos de Caja Negra (Black box) se utilizan en el diseño de casos de prueba relevantes.
   - Las pruebas se desarrollan teniendo en cuenta los requisitos funcionales (establecidas en las especificaciones, conceptos, casos de estudio, reglas de negocio o documentos relacionados).



idat

Tipos de Pruebas de Software

1. **Pruebas funcionales o de caja negra**

   Ámbito de aplicación:

   - Las pruebas funcionales se pueden llevar a cabo en todos los niveles de pruebas.

   Ejecución:

   - El objeto de prueba es ejecutado utilizando combinaciones de datos de prueba derivados/generados a partir de los casos de prueba.
   - Los resultados de la ejecución de la prueba son comparados con los resultados esperados.

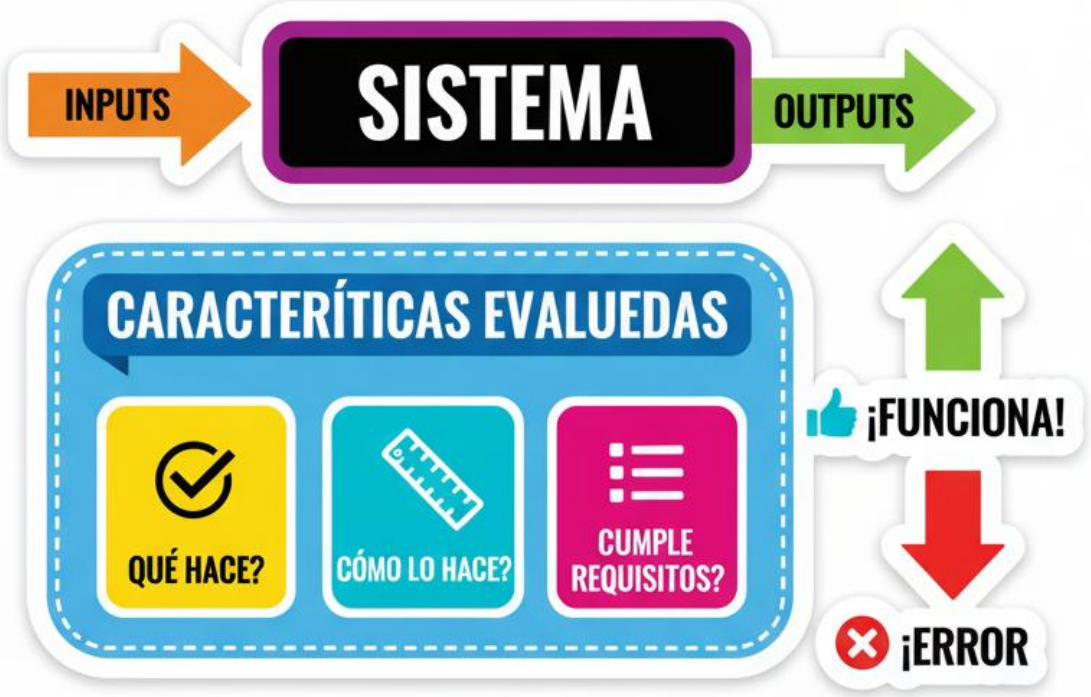
idat

Tipos de Pruebas de Software

2. **Pruebas no funcionales**

   Objetivo: probar las características del producto software

   - ¿De qué forma el software lleva a cabo la función?
   - Las características de calidad no funcionales (fiabilidad, usabilidad, eficiencia, mantenibilidad, portabilidad) a menudo son vagas, incompletas o inexistentes, dificultando las pruebas asociadas a las mismas



idat

Tipos de Pruebas de Software

2. Pruebas no funcionales

Ámbito de aplicación:

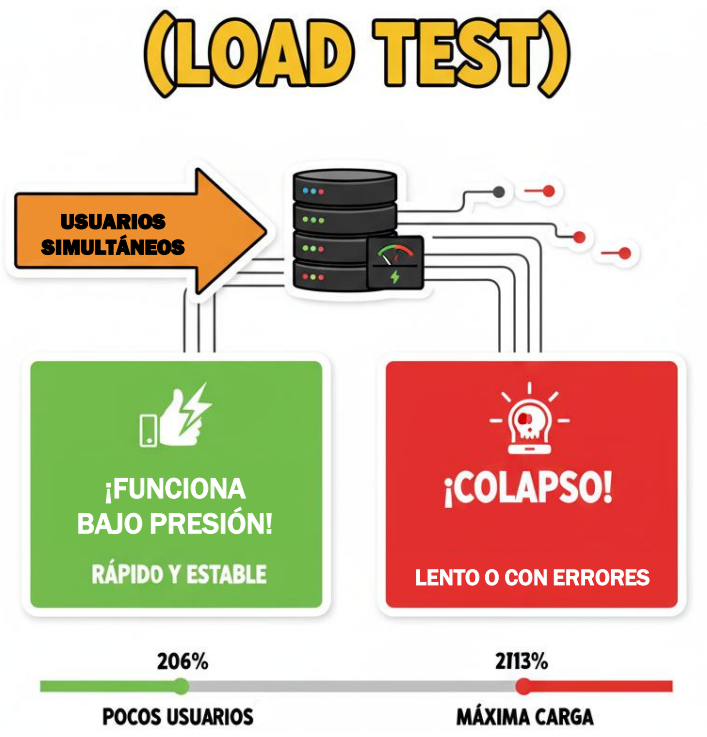
- Las pruebas no funcionales se pueden llevar a cabo en todos los niveles de pruebas.

Ejecución:

- La conformidad con los requisitos no funcionales se mide utilizando requisitos funcionales (seleccionados).

2.1 Prueba de carga (Load test)

- Sistema bajo carga (carga mínima, más usuarios/transacciones).



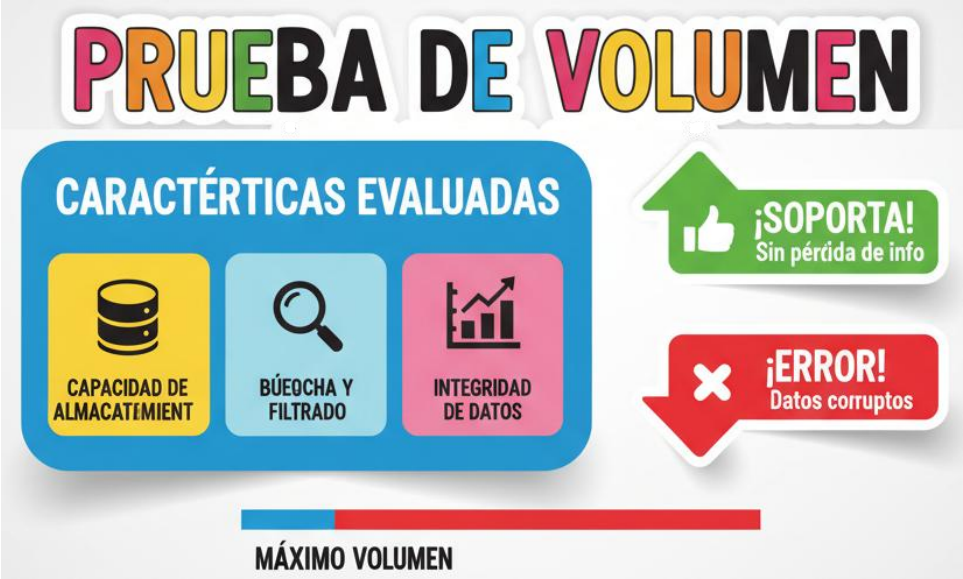
idat

Tipos de Pruebas de Software

2. Pruebas no funcionales

2.2 Prueba de rendimiento (Performance test)

- Rapidez con la cual un sistema ejecuta una determinada función.



2.3 Prueba de volumen (Volume test)

- Procesamiento de grandes cantidades de datos/archivos.

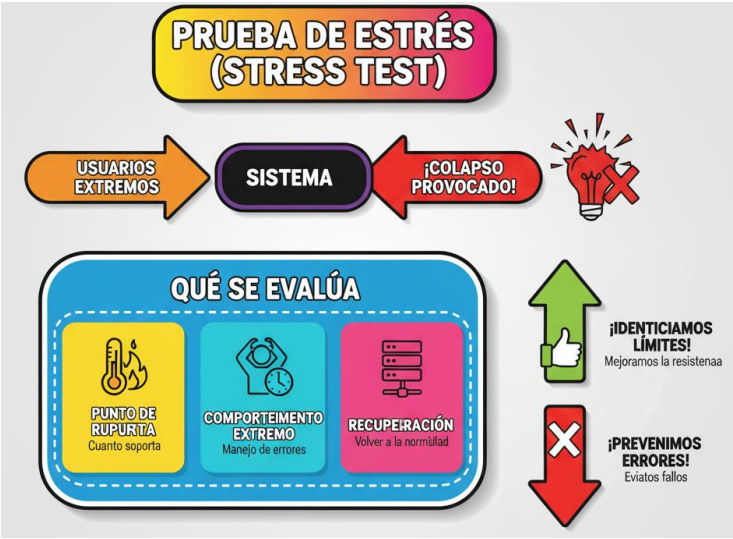
idat

Tipos de Pruebas de Software

2. Pruebas no funcionales

2.4 Prueba de estrés (Stress test)

- Reacción a la sobrecarga/recuperación tras el retorno a una carga normal.



2.5 Prueba de estabilidad (stability test)

- Rendimiento en “modo de operación continua” (tiempo de operación sin reporte de incidencias).

idat

Tipos de Pruebas de Software

2. Pruebas no funcionales

2.6 Prueba de robustez

- Reacción a entradas erróneas o datos no especificados.
- Reacción a fallos de hardware/recuperación ante situaciones de desastre.

2.7 Pruebas de seguridad para los datos del software (Test for data security)

- Protección contra accesos no autorizados.
- Protección contra el robo y daño de datos.

idat

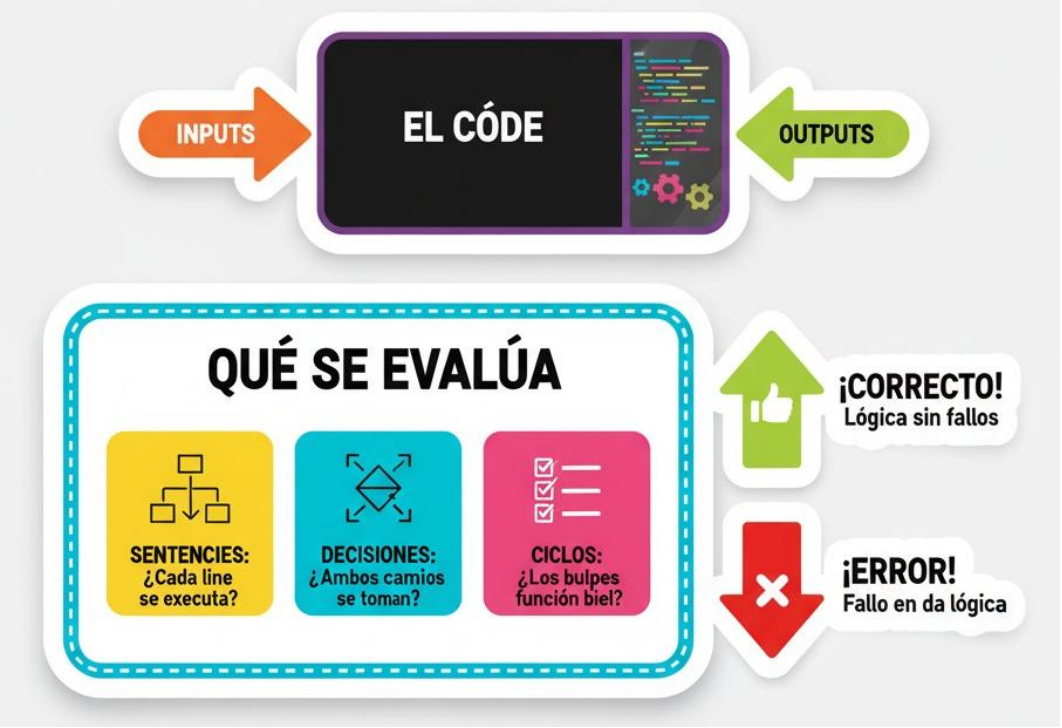
Tipos de Pruebas de Software

2. **Pruebas no funcionales**
   - **2.8 Pruebas de portabilidad/compatibilidad (Compatibility test)**
     - Reacción a distintos entornos (HW, OS, etc.)
   - **2.9 Pruebas de usabilidad (test for usability)**
     - Estructurado, comprensible, fácil de aprender para el usuario.
   - **2.10 Pruebas de mantenibilidad (Maintainability test)**
     - Capacidad de ser analizado y modificado.

idat

Tipos de Pruebas de Software

3. **Pruebas estructurales**
   Objetivo: probar la cobertura de la estructura del producto
   - Análisis de la estructura de un objeto de prueba (enfoque: caja blanca).
   - La finalidad de las pruebas es medir el grado en el cual la estructura del objeto de prueba ha sido cubierta por los casos de prueba.



idat

Tipos de Pruebas de Software

3. **Pruebas estructurales**

Ámbito de aplicación:

- Las pruebas estructurales son posibles en todos los niveles de pruebas, se realizan de forma conjunta a las pruebas de componente y de integración mediante el uso de herramientas.
- El diseño de pruebas estructurales se finaliza tras haber sido diseñadas las pruebas funcionales, con el propósito de obtener un alto grado de cobertura.

Ejecución:

- Se probará la estructura interna de un objeto de prueba (por ejemplo, el control de flujo en el interior de un componente, el flujo a través de la estructura de un menú).
- Todos los elementos estructurales identificados deben estar cubiertos por casos de prueba.

idat

Tipos de Pruebas de Software

Ejemplo de prueba estructural

```
public static void search (int key, int[] elemArray, Result r) {
        int bottom= 0;
        int top =elemArray.length-1;
        int mid;
        r.found=false; r.index=-1;
        while (bottom <=top) {
            mid = (top+bottom)/2;
            if (elemArray[mid] == key) {
                r.index=mid; r.found=true;
                return;
            }
            else {
                if (elemArray[mid]<key)
                    bottom =mid+1;
                else
                    top=mid-1;
            }
        }
    }
```

elemArray	key	r
17	17	(true,0)
17	0	(false,-1)
17,21,23,29	17	(true,0)
9,16,18,30,31,41,45	45	(true,6)
17,18,21,23,29,38,41	23	(true,3)
17,18,21,23,29,33,38	21	(true,2)
12,18,21,23,32	23	(true,3)
21,23,29,33,38	25	(false,-1)

El ejemplo Java, consiste en una función de búsqueda binaria que busca el entero key dentro del elemArray, retornando en el objeto r un campo booleano r.found que indica si ha sido encontrado, y un campo r.index que retorna la posición del elemento encontrado.
Las 8 pruebas que se proponen hacen que todas las sentencias del código se ejecuten al menos una vez, y que en todas las sentencias condicionales se elijan cada una de las dos opciones que tienen.

idat

Tipos de Pruebas de Software

4. **Pruebas de confirmación/regresión**

   Objetivo: probar el objeto de prueba después de cambios

   - Después de que un objeto de pruebas o el entorno del sistema ha sido objeto de modificación todos los resultados de pruebas anteriores resultan inválidos, por lo que las pruebas deben ser repetidas.
   - Es necesario volver a probar el elemento modificado y las zonas adyacentes debido a efectos colaterales no deseados de la funcionalidad cambiada.
   - El alcance de las pruebas de regresión depende del riesgo que la nueva funcionalidad implementada (extensión o corrección de errores) impone al sistema.

idat

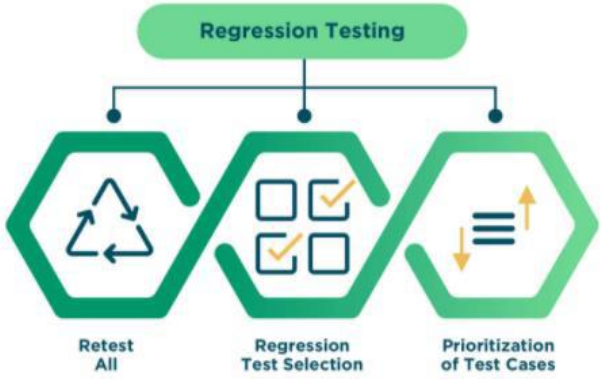
Tipos de Pruebas de Software

4. Pruebas de confirmación/regresión

idat

Tipos de Pruebas de Software

4. **Pruebas de confirmación/regresión**
   Ámbito de aplicación:
   - Las pruebas de confirmación/regresión pueden ser realizadas en todos los niveles de pruebas.
   - Las pruebas típicas tras una modificación son las siguientes:
     - Pruebas de confirmación (Retest). Pruebas para confirmar que el defecto ha sido corregido.
     - Pruebas de regresión. Pruebas para descubrir nuevos defectos introducidos en funcionalidades previamente sin fallos.



idat

Tipos de Pruebas de Software

4. **Pruebas de confirmación/regresión**
   Ejecución:
   - Básicamente la ejecución tiene lugar de la misma forma en la cual se han ejecutado las pruebas en iteraciones previas.
   - En la mayoría de los casos, una prueba de regresión completa no es viable dados sus altos costos y duración.
   - Criterios para la selección de casos de prueba de regresión:
     - Casos de prueba de prioridad alta (rutas o flujos funcionales críticos).
     - Probar solamente la funcionalidad estándar, saltarse casos y variaciones especiales.
     - Probar solamente la configuración utilizada con mayor frecuencia.
     - Probar solamente subsistemas/zonas seleccionadas del objeto de pruebas que tienen el riesgo de sufrir alteraciones.

idat

Tipos de Pruebas de Software

5. **Pruebas de mantenimiento**

El cliente ha probado el producto y es puesto en producción.

- El ciclo de desarrollo inicial, incluidas las pruebas asociadas, ha sido completado.

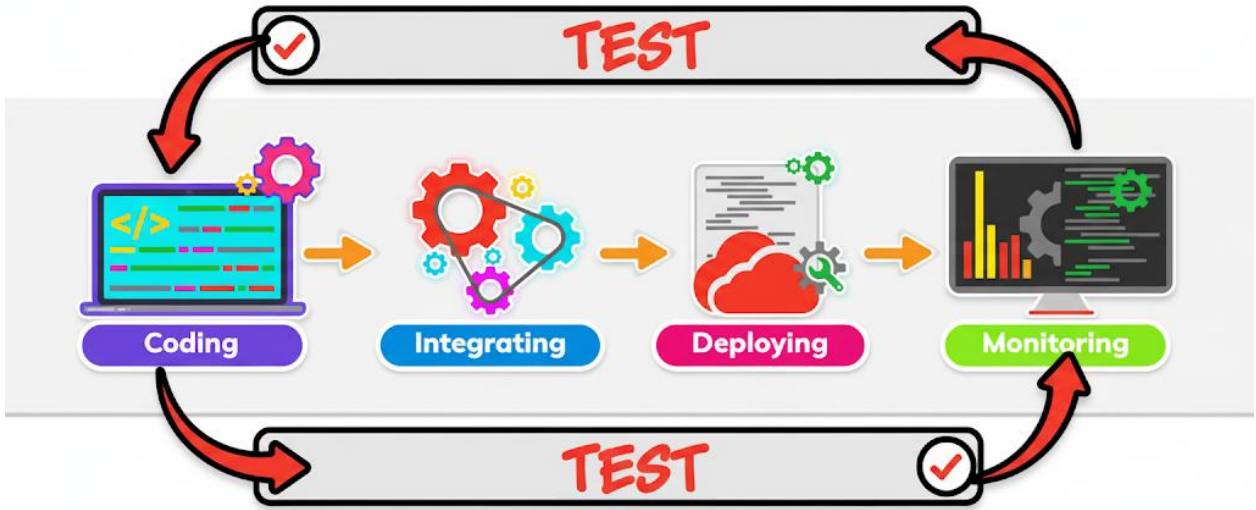
El mismo software se encuentra al comienzo del ciclo de vida.

- Será utilizado por muchos años, será ampliado.
- Es muy probable que aún contenga defectos, por lo tanto, será modificado y corregido.
- Necesitará adaptarse a nuevas condiciones y deberá integrarse a nuevos entornos.

idat

Tipos de Pruebas de Software

5. **Pruebas de mantenimiento**
   Pruebas de regresión:
   - Las pruebas de regresión son necesarias y realizadas una vez realizado algún cambio en el sistema.
   - El alcance de las pruebas de regresión puede variar en función de las circunstancias (condiciones del entorno).



Cualquier nueva versión del producto, cada nueva actualización y cada cambio del software requiere pruebas adicionales

idat

Conclusiones

- Utilizar todos los niveles de prueba de manera estratégica permite construir software de alta calidad, reduciendo costos (al encontrar errores pronto) y aumentando la confianza para el despliegue.

idat

¿Dudas?

¿Miedos?

¿Temores?

idat

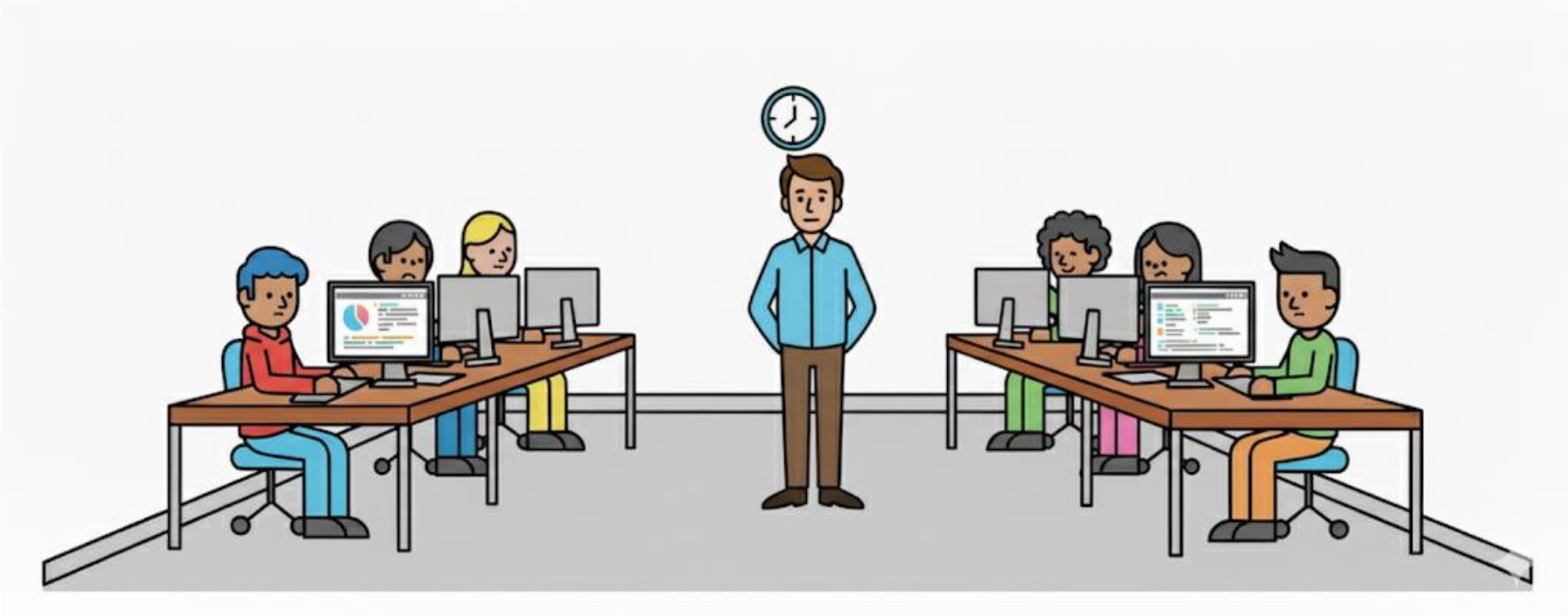
Verificamos lo aprendido:

Kahoot!

http://www.kahoot.it/

idat

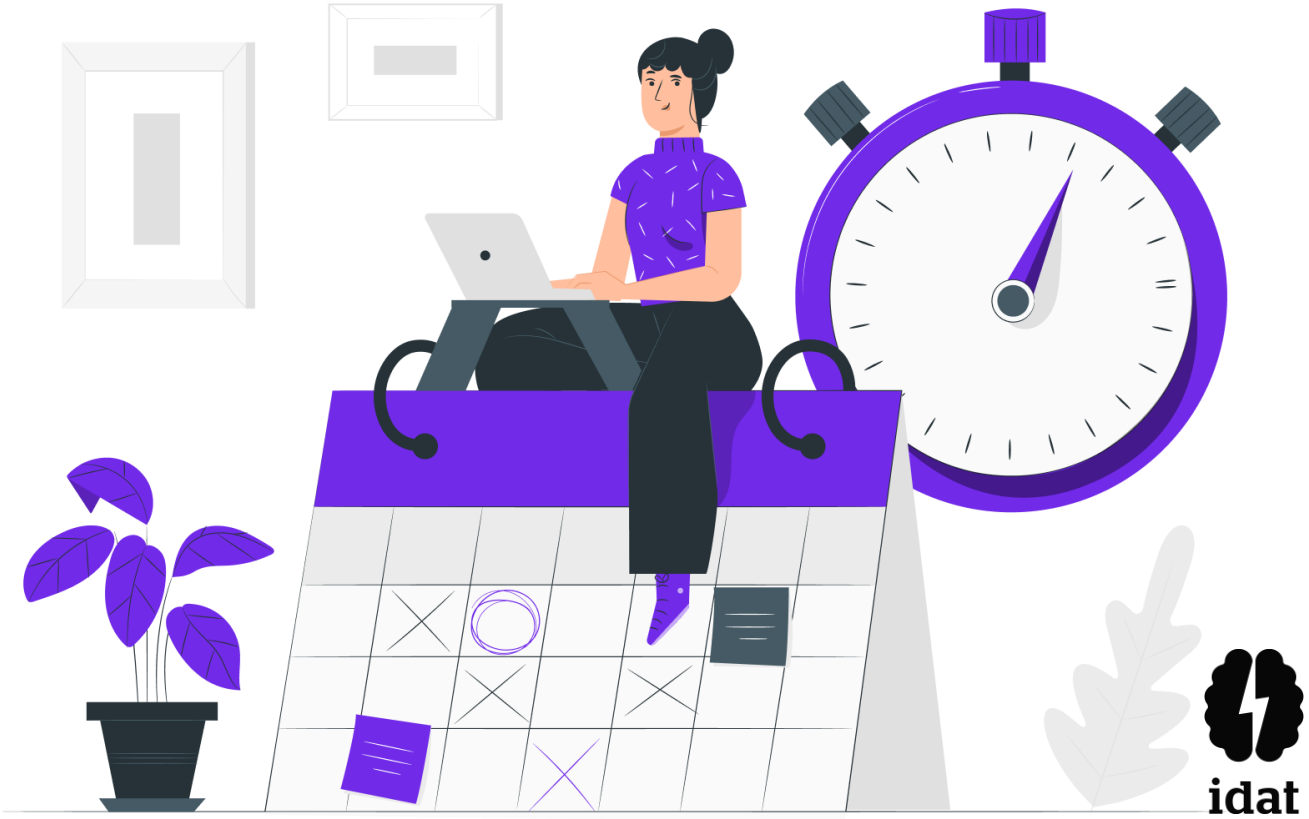
Primera Evaluación Continua



idat

Aplicamos lo aprendido: Asignación para la clase siguiente

Resolver la actividad en el EVA



idat

idat

GRACIAS